Warden32 — User Guide

Version 0.3.0 · MIT · completely offline (no network, no telemetry)

Warden32 is a deterministic browser lab for studying what an ESP32-based guest-network firewall/NAT router (ESP32-WROOM-32D + LAN8720, stock ESP-IDF Wi-Fi + lwIP) can and cannot enforce. You build a small network, give IoT devices behaviour, attack it, watch the real evaluation pipeline decide every flow, and export reproducible artifacts.

Every number in the UI is tagged with its provenance — measured, datasheet, paper, estimate or assumption — and anything the firmware cannot enforce is labelled not enforceable instead of quietly pretending to block it. This guide teaches the UI; the deep engineering docs are linked at the end.

---

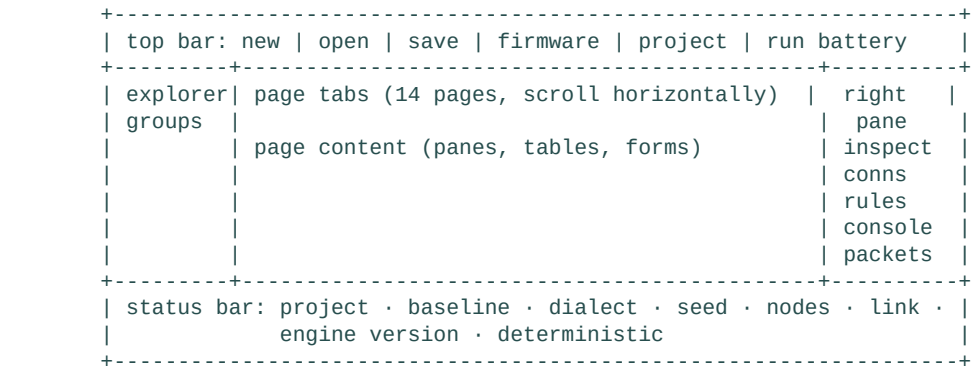
1. Install and run

Requirements: Node.js 20+ (developed on v26), any modern browser. No network access is required at runtime; everything runs locally.

```
npm install
npm run dev        # http://localhost:5173 — development server
npm run typecheck  # tsc --noEmit
npm test           # 144 deterministic tests
npm run build      # typecheck + production build into dist/
npm run preview    # serve the production build
```

Opening dist/index.html via file:// also works. Projects are stored as JSON (.warden32); the app never sends data anywhere.

2. The shell



- Explorer (left) — devices grouped by zone; click any device to inspect it. Groups mirror the sample project: INTERNET, ATTACKER, VENDOR CLOUD, C2 SERVER, DNS RESOLVER, NTP SERVER, UPSTREAM LAN, HOME ROUTER, GUEST WI-FI, ESP32 ROUTER, TRUSTED LAN, MANAGEMENT.

- Right pane — five tabs: inspect (selected device), connections (conntrack table), rules (hits per rule), console (sim event stream), packets (per-flow pipeline trace, 5 stages).

- Status bar — current project, experiment baseline (P-A/P-B/P-C), firewall dialect, seed, device count, WAN link state, engine version.

"deterministic" reminds you the same inputs always replay identically.

Deep links: ?page=<id> opens a specific page (overview, topology, devices, activity, traffic, rules, attacks, incident, resources, experiments, compare, hardware, registry, docs). Activity adds &run=activity, &run=activity-normal or &run=activity-compromised to run immediately on load — useful for demos and screenshots.

3. Projects and firmware

- new — restores the deterministic sample lab (seed 1337): public WAN 192.168.1.23, guest Wi-Fi 10.0.0.0/24, three managed IoT devices, uplink locked infrastructure devices, and a few sample flows.

- save / open — writes/reads .warden32 JSON. Drag a file onto the window to open it. Validation runs on load; non-blocking findings are shown as warnings, structural errors refuse the file.

- firmware — switches the simulated build:
  - stock-eth / stock-wifi (ESP-IDF stock, 4-list ACL, default-ALLOW);
  - softap-sta (SoftAP + STA, no filtering at all);
  - target-v1 (the stateful reference dialect used as the design target).

Switching resets results honestly (old verdicts would be stale).

- run battery (>) — reruns the built-in sample flow set.
- Autosave — the current project is saved to localStorage continuously and reloaded on start. If you upgraded from an older version and the page looks stale, press new (or open a file) — the old autosave is still a valid project, it just keeps its old devices.

4. Devices and bandwidth (M10)

Open the devices page. You need it for two M10 features: adding IoT devices and configuring bandwidth.

4.1 Add a device

[add device] pane:

- archetype — 17 IoT archetypes (IP camera, smart plug, thermostat, doorbell, printer, TV, speaker, sensor, hub, lock, baby monitor, robot vacuum, e-reader, console, ...). Each carries a default traffic profile and a default demand in Mbps.

- zone — guest (the Wi-Fi AP), trusted or mgmt. Interfaces of trusted/mgmt start DOWN in the sample; activity is dropped for devices on down links (with a note), so bring the interface up first on the topology page if you add devices there.

- name — free text; the id is a slug plus a suffix.
- demand Mbps — optional override of the archetype default volume.
- cap Mbps — optional per-device cap (see 4.3).

The device gets the next free 10.x.0.x address from its zone and a fresh 3c:71:bf:... MAC automatically. Addresses never collide with existing devices, interface addresses, or the DHCP pool range.

4.2 The managed table

Columns: device (id, archetype, traffic profile), zone, ipv4, iface, demand Mbps, cap Mbps, state, actions.

- demand — commit on blur/Enter; empty restores the archetype default (shown as default x.xx). This is the volume the activity page will try to deliver over the selected window.

- cap — a configured *target*, committed on blur/Enter, empty = none. When demand exceeds the cap the row shows a demand above cap chip.

- state — online / offline (engine-level control; resets when the engine rebuilds) and a compromised checkbox (persists in the project file; compromised devices run the compromised activity profile).

- actions — remove for removable IoT devices only. The ESP32 router, gateways and external hosts are protected. Removing a device also cleans up port forwards and rules that reference it.

- mark all compromised / clear compromised buttons set the flag for every managed IoT device at once — handy for before/after demonstrations.

4.3 Honesty: caps are not enforceable

The stock 4-list ACL has no rate limiting and no fairness concept, so a per-device Mbps cap cannot be enforced by the firmware this lab simulates. The cap is a configuration target; the activity page reports it as not enforceable and shows demand vs cap so you can see how far the target is from reality. (The stateful target-v1 dialect could enforce packet-rate rules, but still not Mbps shaping — the note on the activity page reflects the active dialect.)

4.4 Uplink rate

[uplink] bandwidth available to the ESP32 sets the WAN link rate used for capacity accounting (default 100 Mbps — the LAN8720 is a 100 Mbit/s PHY, datasheet). Two different ceilings matter:

- the link rate (what the PHY carries), and
- the routed ESP32 throughput estimate, 5–15 Mbps (estimate from literature) — the router becomes the bottleneck long before the link does.

The uplink number does not shape traffic: queueing is not simulated, and the model says so. Instead, the activity page reports total demand, saturation percentage and an over-subscription warning compared against both ceilings.

5. Activity — normal and compromised IoT behaviour (M10)

Open the activity page.

5.1 What a run does

For every managed IoT device (role iot, zones guest/trusted/mgmt, not the router), the engine:

1. picks the activity set — normal profile from the device's traffic profile, or the compromised set (C2 beaconing, DNS tunnelling, LAN scanning,

exfiltration) when the device is compromised or the mode forces it;

2. sizes each activity from the device's demand within the chosen window;
3. schedules flows with seeded jitter (mulberry32, seed from the field);
4. submits every flow through the real pipeline — the same classify -> offline -> quarantine -> policy -> NAPT -> route -> verdict path as manual traffic. Verdicts, drops, NAT state and events are real engine results;

5. advances the engine clock to the end of the window.
Determinism: same project + same seed + same window = byte-identical flows, every time, in every browser. Consecutive runs continue the engine clock (a run advances time — that is the point).

5.2 Controls

- window — 5 minutes, 1 hour, 6 hours or 24 hours of simulated time.
- mode — use device states (compromised devices run compromised profiles, the rest normal), force normal, force compromised.

- seed — integer; defaults to the project seed.
- run activity — executes and stores the report. Device or bandwidth edits clear a stale report; results are never silently kept.

5.3 Reading the report

- summary — flows, verdict counts (allowed/denied/dropped/not-enforceable), MB attempted vs allowed, average consumed Mbps, and the engine clock after the run.

- capacity — uplink rate, total demand, total consumed, saturation meter, the 5–15 Mbps router ceiling, and the per-device-caps-not-enforceable note with its dialect-aware explanation. over-subscribed appears when demand exceeds the link rate.

- per-device accounting — demand, cap, attempted Mbps, consumed Mbps (verdict allowed only), peak Mbps (busiest simulated minute), sent MB, received MB, and verdict chips per device.

- per-activity breakdown — every activity with its target, flow count and allowed bytes.

- notes & export — the honesty notes for the run plus export JSON, export CSV and export markdown (same report shaped for spreadsheets and papers).

5.4 Sampling honesty

A 24-hour camera window would be tens of thousands of real events; the engine generates *sampled representations*: each generated flow aggregates sampleOf raw events and its byte size is the summed volume. Byte totals equal the configured demand (the accounting is exact); flow counts are samples, not packet traces. The report says this explicitly. Similarly, peak tracks the average closely because samples are spread evenly — no fabricated burst structure.

5.5 What the sample project shows

With the sample lab (uplink 100 Mbps, camera demand 2.5 Mbps, plug/thermostat

0.05 Mbps) a 1-hour use device states run produces roughly 720 flows and 1.17 GB attempted at 2.6 Mbps average — about 2.6% of the link, well under even the pessimistic 5 Mbps router ceiling.

Now tick the camera's compromised checkbox on the devices page and re-run with use device states: the camera's activity switches to C2 beacons (203.0.113.66:8443), DNS tunnelling (203.0.113.53:53), LAN scanning (10.0.0.0/24:445) and exfiltration (203.0.113.65:443). Note that with the stock firmware and default-ALLOW these flows are allowed — that is the finding, not a bug: the stock ACL cannot rate-limit or meaningfully prevent egress, and caps remain unenforceable. Quarantine the device on the incident page (or add deny rules on rules) and re-run to watch the same profile get dropped.

5.6 Shareable deep links

- ?page=activity&run=activity — run in "use device states" mode on load
- ?page=activity&run=activity-normal — force normal
- ?page=activity&run=activity-compromised — force compromised

The run is a normal deterministic run; the page just doesn't wait for your click. Documented, not hidden.

6. The other pages (short tour)

- overview — lab at a glance: pipeline sketch, honesty ledger, quick controls, recent events. Good first stop.

- topology — the kit-19 canvas: zones, devices, links, the bypass arc (traffic that never touches the router), RMII/reference details. Everything is draggable (positions persist per project; # resets). Click a node to inspect; toggle interfaces and links here.

- traffic — manual flow lab: pick a device, a behaviour state (normal/noisy/misconfigured/compromised), a target and a profile; submit single flows or small batches and read the 5-stage trace in packets.

- rules — edit stateful rules and the stock 4-list ACL per interface, port forwards, default policies. Rule hit counters appear in the right pane rules tab. The stock dialect refuses rule types it cannot express.

- attacks — 12 scenarios (ARP spoof, rogue DHCP, DNS spoof, lateral movement, C2 beacon, exfiltration, deauth — wifi-deauth and dhcp-starvation are explicitly *not modeled* and say so). Run one, inspect, then respond.

- incident — incident log from attack runs; quarantine/release devices; addBlockRule (stock honestly refuses: it cannot block a source host).

- resources — RAM (180 KB base + per-flow costs), CPU window, airtime, TX queue, conntrack/NAPT occupancy; 7 failure injections; link/device toggles. Everything is estimate-tagged.

- experiments — baselines P-A (stock, permissive), P-B (stock, rules), P-C (target-v1 stateful), plus the P-D label; run the suite, compare

results, export JSON/CSV/Markdown with provenance and engine version.

- compare — side-by-side verdict matrices across baselines/experiments.
- hardware — the validation checklist: everything you must measure on a real board to replace estimate placeholders. Never auto-fills values.

- registry — boards, ethernet adapters, firmware profiles, archetypes; the provenance of everything the sim can instantiate.

- docs — in-app index of the documentation set.

7. Exports and reproducibility

All exports are stamped with tool name, engine version and (where relevant)

project name/seed; imports are versioned and validated.

- warden32-activity.json — full activity report (devices, activities, capacity, notes) minus per-flow results.

- warden32-activity.csv — one row per device and per activity.
- warden32-activity.md — paper-ready tables and notes.
- Project files .warden32 — full state; deterministic replay via seed.
- Experiment exports — JSON/CSV/Markdown including provenance tables.

To reproduce someone else's result: same project file, same seed, same

window, same engine version (shown in the status bar). See

docs/REPRODUCIBILITY.md.

8. FAQ / troubleshooting

- The page shows old data after an upgrade. Autosave keeps your old project. Press new to load the current sample (this guide assumes it), or

open your saved file — both are valid.

- Activity report is empty. No run yet in this session; results are in-memory by design (exports are the way to keep them). Press

run activity.

- "no IoT devices in managed zones" notice. Add a device on the devices page, or move an existing device into guest/trusted/mgmt.

- Everything is "not enforceable". The stock ACL has no rate limits and no fairness. Switch to target-v1 on the hardware page to see the

stateful dialect story (packet-rate rules exist, Mbps shaping still ends at

the ACL).

- My compromised device's flows are still allowed. Default-ALLOW plus no deny rules = allowed. Quarantine it or add rules, then re-run.

- Clock cannot move backwards. Consecutive runs advance simulated time; reports reflect engine time, not wall clock.

- Screenshots look stale. Browser profile/localStorage reuse; press new.

More: docs/TROUBLESHOOTING.md.

9. Limitations (read this before citing numbers)

- Not a packet-level bench: no lwIP emulation, no radio/timing/thermal physics; no queueing model (over-subscription is reported instead).

- RAM/CPU/airtime and the 5–15 Mbps routed throughput are seeded estimates

calibrated from literature, not measurements.

- Caps are targets; the simulated firmware cannot enforce Mbps shaping.
- Attack coverage is explicit, not complete; unmodeled techniques stay labelled.
- Hardware validation is pending: no measured columns are populated until you produce them with docs/HARDWARE-CHECKLIST.md.

Full list: docs/LIMITATIONS.md.

10. Documentation map

doc	what it covers
ARCHITECTURE.md	module map, storage, determinism
PIPELINE.md	every pipeline stage in order
FIREWALL.md	dialects, ACL semantics, what is enforceable
VERIFIED-FACTS.md	sources behind every claim
EXPERIMENT-METHODOLOGY.md	baselines, suite, metrics
REPRODUCIBILITY.md	seeds, versions, replay recipes
TEST-PLAN.md	what the 144 tests cover
EXTENDING.md	adding archetypes/profiles/attacks
HARDWARE-CHECKLIST.md	real-board validation procedure
LIMITATIONS.md	honest capability boundary
TROUBLESHOOTING.md	common symptoms and fixes
DECISIONS.md	dated design decisions log

Warden32 is defensive research tooling. Use the attack scenarios only on networks and hardware you own or are authorized to test (SECURITY.md).